Maltego Introduction – Creating a simple local Transform

Author

Joas Antonio dos Santos

https://www.linkedin.com/in/joas-antonio-dos-santos/

TABLE OF CONTENTS

01 WHAT IS MALTEGO

04 MALTEGO LIBRARIES AND FRAMEWORKS

08 CREATING A SIMPLE LOCAL TRANSFORM

WRITTEN BY JOAS A SANTOS

WHAT IS MALTEGO

INTRODUCTION

Maltego is an advanced open source intelligence and forensic analysis tool widely used to collect and connect information during cyber investigations. It is especially popular among information security professionals for analyzing networks and vulnerabilities, as well as collecting information about specific targets such as organizations or individuals.

On my journey as a Red Team professional, maltego is a tool that I use frequently, not only for OSINT, but for Threat Intelligence projects and research that I do in my free time, just to pass time. And during some studies I was developing a Transform to assist in human trafficking and missing persons investigations and I decided to create this simple PDF so that other enthusiasts can create their own Transforms and not just depend on the paid ones or those on the Transform Hub.

Maltego features:

1. Transforms: Transformations are the main feature of Maltego, allowing users to process information from various data sources and generate interactive graphs. Transformations can extract, correlate, and visualize data from public or private sources.

2. Integration with external sources: Maltego can integrate with a wide variety of data sources, including public records, WHOIS databases, social networks, geolocation APIs, and more. This allows users to obtain a large amount of information without having to manually visit each source.

3. Data Visualization: Maltego is highly effective at visualizing complex information networks. The data is displayed in a graphical format that shows connections between entities such as people, groups, internet domains, and others.

Comms Server

A Comms server on Maltego allows multiple users to work interactively on a shared graph in real time. Users can interact via an integrated chat messenger and also send links to selected parts of the chart. Shared graphics are kept private with a session key that encrypts communication traffic using 128/256-bit AES encryption. Shared graph sessions are compatible across different platforms, allowing all four different clients to join the same shared graph.

What is CTAS

CTAS (Commercial Transform Application Server) is a server that hosts all of Maltego's standard transformations and executes these transformations as requested through the Maltego desktop client. It is designed for companies that want to keep their transformation requests private by hosting them internally. This is useful for sensitive investigations where it is preferable that data not transit through the Maltego infrastructure. The CTAS server is delivered as a Docker image and requires Internet access to connect to various online sources.

What is TDS and iTDS

The public TDS (Transform Distribution Server) is a server located in the Maltego infrastructure, freely available and used to write remote transformations. iTDS (internal Transform Distribution Server), on the other hand, offers the same functionality as public TDS, but can be hosted internally on an organization's own infrastructure. This is ideal for handling sensitive internal data that should not pass through the Internet or external infrastructure.

About TDS

A Transform Distribution Server (TDS) in Maltego allows you to combine transformations, entities, machines and their configurations into a single item that can be distributed and installed by different Maltego users. This makes it easy to share custom transformations and configurations among a team of analysts, or with the world if you wish. In TDS, you can manage custom transformations, configurations, entities, and perform backups. It is useful for those who want to integrate their data into Maltego by creating custom transformations.

How does it work?

This diagram shows the architecture of Maltego and how it interacts with different data sources and servers. End users use various versions of Maltego (Classic, XL, CE) that connect to the internal TDS server (iTDS) via an API. iTDS allows administrators and developers to manage custom transformations and configurations. These transformations can query public data sources (such as DNS, social networks, search engines) or internal data from the organization (internal APIs, system logs, internal servers). Configurations and transformations are managed through a server interface, which can also be used to make backups.

Maltego Server Architecture

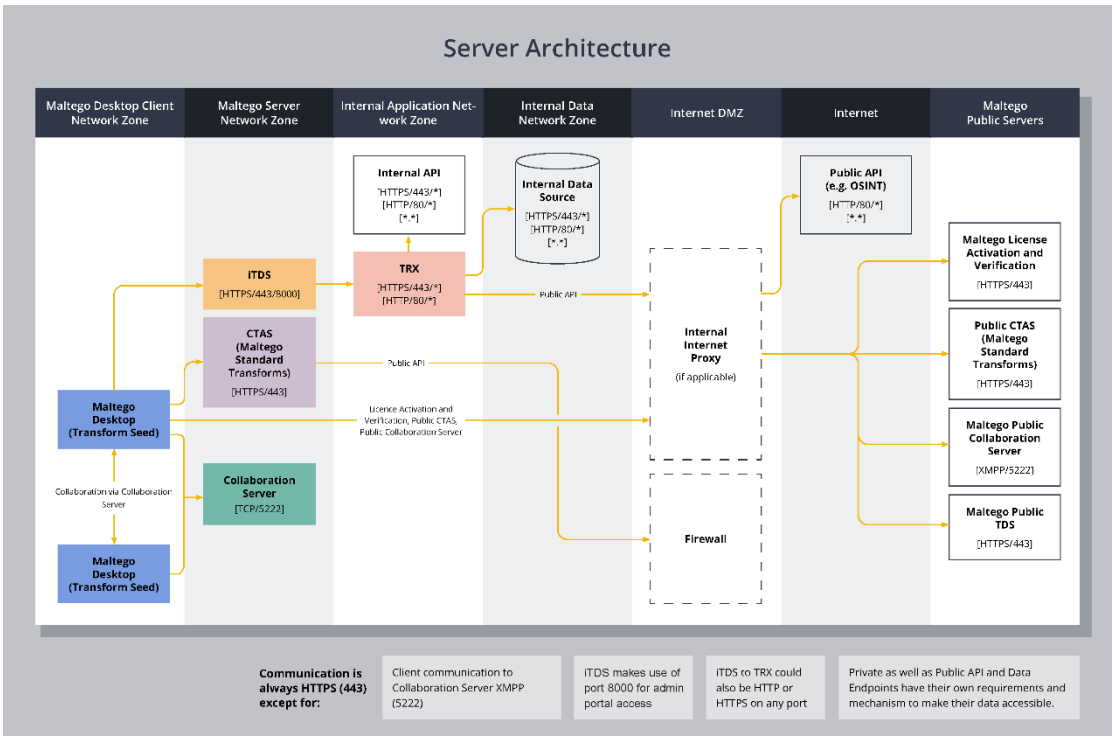

Figure 1 – Server architecture (docs.maltego.com)

The Maltego server architecture illustrated in the image details how different components interact within a firewall-protected network environment. Includes:

- **Network Zones**: Different zones such as Maltego client network zone, Maltego server network zone, and internal data and application network zones.
- **iTDS and CTAS**: iTDS serves as the distribution server for internal transformations, and CTAS (Commercial Transform Application Server) hosts the standard Maltego transformations. Both operate under HTTPS for security.
- **TRX and Public APIs**: TRX manages internal and public APIs for data integration, also accessible by other collaboration and license verification services.
- **Proxies and Firewalls**: There are security mechanisms such as firewalls and internal internet proxies that regulate data traffic.
- **Maltego Public Servers**: Includes servers for license activation, public collaboration, and transform distribution, all communicating over HTTPS, except the collaboration server which uses TCP/5222.

This architecture enables secure and efficient communication between Maltego users and data resources, both internal and external.

MALTEGO LIBRARIES AND FRAMEWORKS

INTRODUCTION

Transform developers for Maltego are responsible for creating the logic that translates transformation requests into accessible data. This data can come from a variety of sources, such as databases or APIs. Maltego's desktop application simplifies the management of interconnected graphs, allowing developers to focus on extracting and querying data. To facilitate this process, there are libraries that help host an HTTP server, translate XML requests and manage responses, making interaction with objects more efficient.

Libraries and Frameworks

Libraries and frameworks available for developing transformations in Maltego, using different programming languages:

- **Canari3 (Python)**: A framework for rapid development of local and remote transformations in Maltego, enabling efficient prototyping, packaging and distribution. Supports both local and iTDS transformations.
- **MaltegoGo (Go)**: This library is a translation of Maltego's TDS library to Go, allowing the creation of extremely fast transformations for iTDS, but does not support local transformations.
- **TransNet (.NET)**: A .Net Standard library that makes it easy to create transformations in Maltego, compatible only with iTDS, does not support local transformations.
- **MaltegoLocal (GoLang)**: Local wrapper for developing transformations in Maltego, allowing fast implementations in a local environment, but is not compatible with iTDS.
- **JavaMaltego (Java)**: Library developed in Java to create local transformations in Maltego, focused on developments that do not require integration with iTDS.

WRITTEN BY JOAS A SANTOS

- **Maltego-TRX:**The Maltego TRX library is a Python tool intended for developing transformations in Maltego. It makes it easy to create transformation servers that can interact with data from external sources such as SQL databases or REST APIs. With Maltego TRX, developers can quickly generate transformations and host them on a server that communicates via HTTP/XML. The library also allows you to create and manage transformation projects in an organized way, including a development server that automatically reloads when modifying the code.

How to create a transform server?

To create a transformation server in Maltego, the server must be on a network that can access the required data, such as a SQL server or REST APIs. Any server or virtual machine that exposes port 8080 (or 80) and allows HTTP (XML) traffic can function as a transform server. Maltego TRX, a Python library, requires Python 3 and can be installed via PIP. After installation, you can create a new project and add transformations to the corresponding folder. To start the server in development mode.

Commands for creating the server:

```
$ pip install maltego-trx
```

Install the maltego-trx library in Python

```
$ maltego-trx start <project name>
```

Configure a new project for Maltego and the transforms will be in the folder with the respective name

```
$ python Project.py runserver
```

The above command launches developer mode.

Code structure of a Transform

When creating a Transform for Maltego in Python, the code structure is not that complex.

1. Importing Libraries

First, import all the necessary libraries. Typically this will include the maltego_trx framework for integration with Maltego, and any other libraries required for Transform functionality, such as requests for API calls or beautifulsoup4 for HTML parsing.

```
from maltego_trx.maltego import MaltegoMsg, MaltegoTransform

from maltego_trx.transform import DiscoverableTransform

import requests
```

2. Definition of the Transform Class

Define a class that inherits from DiscoverableTransform or Transform. This class will be the core of your Transform, containing all the logic necessary to perform the intended task.

```
class ExampleTransform(DiscoverableTransform):
```

3. Method for Creating Entities

Implement a method called create_entities that is called by the Maltego framework. This method will receive the input message from Maltego (MaltegoMsg) and the response from transform (MaltegoTransform), which you will use to add entities to the Maltego graph.

```
@classmethod

def create_entities(cls, request: MaltegoMsg, response: MaltegoTransform):

# Logic to process the input and add entities to the response.
```

4. Transform Specific Logic

Inside the create_entities method, implement your Transform's specific logic. This may include calling external APIs, processing data, and creating and configuring Maltego entities to be returned to the user.

```
input_value = request.Value # Get the input value

data = requests.get(f"https://api.example.com/data/{input_value}") # API call

if data.status_code == 200:

json_data = data.json()

entity = response.addEntity('maltego.ExampleEntity', json_data['name'])

entity.addProperty('detail', 'Detail', 'strict', json_data['detail'])

else:

response.addUIMessage("No data found or API error", messageType='PartialError')
```

5. Conditional Execution

Add an if __name__ == "__main__" block at the end of the file to make the script directly executable. This is useful for testing Transform locally.

```
if __name__ == "__main__":
```

```
from maltego_trx.server import serve_transform_classes

serve_transform_classes([ExampleTransform])
```

6. Transform Configuration

After the Python code, you will also need to configure Transform in Maltego, including specifying the command, parameters, and script path. This is usually done through the Maltego interface or a JSON configuration file.

This basic framework provides a skeleton for developing Transforms

CREATING A SIMPLE LOCAL TRANSFORM

INTRODUCTION

Now let's create our local transform, don't forget to have maltego installed and configured on your machine, as well as python3 and the maltego-trx library installed

Creating a simple Transform to search the Tor Network

To create this transform, I will use the Python language and the Ahmia website to perform the scraping and carry out searches using Entity, in this case the best option being “phrase” within Maltego.

What are Entities

In Maltego, "entities" are the visual objects used to represent data within the tool's graphical environment. Each entity can represent different types of information, such as people, organizations, IP addresses, internet domains, among others. These entities are connected by lines or "links" that represent relationships or data flows between them. Entities can be enriched with additional data through transformations, which are scripts or queries that extract information from databases or the internet. These entities are central to visualizing and analyzing complex networks of information within Maltego.Top of form

Putting it into practice:

```
$ maltego-trx start ahmia2
```

Let's create the project with the name Ahmia or whatever you prefer.

Now let's import the libraries to prepare the environment

```
from maltego_trx.maltego import MaltegoMsg, MaltegoTransform

from maltego_trx.transform import DiscoverableTransform

import requests
```

```
from bs4 import BeautifulSoup

from urllib.parse import urlparse, parse_qs, unquote
```

Some libraries may not be available, just install via PIP

```
$ pip install requests bs4 maltego-trx
```

This way you install the libraries necessary for our code to work, detailing more about the libraries

- **MaltegoMsg and MaltegoTransform**: Maltego_trx classes that are used to manipulate the Maltego input message and construct the response to be sent back to Maltego respectively.
- **DiscoverableTransform**: Base class that facilitates the creation of Transforms that can be discovered and managed by the maltego_trx framework.
- **requests**: Library for making HTTP requests in a simple and readable way in Python.
- **BeautifulSoup**: Library for parsing HTML and XML documents, used to extract information from web pages.
- **urlparse, parse_qs, unquote**: Functions from the urllib.parse module that are used to parse and manipulate URLs.

```
class AhmiaDomainExtractor(DiscoverableTransform):

@classmethod

def create_entities(cls, request: MaltegoMsg, response: MaltegoTransform):

search_term = request.getProperty('text')

html_content = cls.search_ahmia(search_term)

unique_domains = cls.parse_results(html_content)



if unique_domains:

for domain in unique_domains:

entity = response.addEntity('maltego.Domain', domain)

else:
```

```
entity = response.addEntity('maltego.Phrase', 'No domains found')

entity.addProperty(fieldName="description", displayName="Description", value="Search returned no results")
```

- **create_entities method**: This method is automatically called by the maltego_trx framework when Transform is executed. It processes the incoming message, performs the search in Ahmia and extracts the domains, adding them to the Maltego graph or indicating that no results were found.

- **@classmethod decorator:**This decorator indicates that the method is a class method, which means that it operates against the class and not a specific instance of it. This is useful for operations that do not require an object of the class to be performed.

- request is a MaltegoMsg object that contains all the message data sent by the Maltego client. It allows you to access properties and values that were passed through the Maltego interface.

- response is a MaltegoTransform object that is used to construct the Transform response. Here you add entities to the Maltego graph, configure error or information messages, among others.

- cls.search_ahmia(search_term): Calls the static method search_ahmia which makes an HTTP request to the Ahmia website using the search term. Returns the HTML content of the results page.

- cls.parse_results(html_content): Parses the returned HTML content to extract the domains of the URLs found, using the parse_results function.

```
@staticmethod

def search_ahmia(search_term):

base_url = "https://ahmia.fi"

search_url = f"{base_url}/search/?q={search_term}"

try:

response = requests.get(search_url)

response.raise_for_status()

return response.text

except requests.exceptions.RequestException as e:
```

```
print(f"Request failed: {str(e)}")

return None
```

- **search_ahmia method**: Performs a GET request to Ahmia using the provided search term. If the request is successful, it returns the HTML content of the results page. Otherwise, it catches and prints exceptions related to the HTTP request.

```
@staticmethod

def parse_results(html_content):

soup = BeautifulSoup(html_content, 'html.parser')

domains = set()

for result in soup.select('h4 > a'):

if result.get('href'):

parsed_href = urlparse(result['href'])

redirect_params = parse_qs(parsed_href.query)

redirect_url = redirect_params.get('redirect_url', [None])[0]

if redirect_url:

domain = urlparse(unquote(redirect_url)).netloc

domains.add(domain)

return domains
```

- **parse_results method**: Parses the HTML returned by Ahmia to extract redirect URLs and subsequently the domains of the redirected URLs. Uses the BeautifulSoup library to parse HTML and extract specific elements (h4 > a). Domains are added to a set to avoid duplicates, and this set is returned.

These explanations describe the logic behind the code, showing how Transform interacts with the Ahmia website, processes data, and manipulates results within the Maltego environment.

Full source code link:
https://github.com/CyberSecurityUP/AhmiaDomainExtractor-Maltegoce/

Configuring and Installing Transform Local on Maltego

```
┌──(root㉿packt-evasion)-[/home/packt]
└─# maltego-trx start ahmia
Successfully created a new project in the 'ahmia' folder.
```

Figure 2 – Creating the project

I created the Ahmia project with maltego-trx, if everything goes well it will create the project successfully, check if you have the necessary permissions

```
┌──(root㉿packt-evasion)-[/home/packt/ahmia]
└─# cd transforms

┌──(root㉿packt-evasion)-[/home/packt/ahmia/transforms]
└─# nano AhmiaDomainExtractor.py

┌──(root㉿packt-evasion)-[/home/packt/ahmia/transforms]
└─# cd ..

┌──(root㉿packt-evasion)-[/home/packt/ahmia]
└─# python project.py list
= Transform Server URLs =
/run/ahmiadomainextractor/: AhmiaDomainExtractor
/run/dnstoip/: DNSToIP
/run/greetperson/: GreetPerson
/run/greetpersonlocalized/: GreetPersonLocalized
/run/overlayexample/: OverlayExample


= Local Transform Names =
ahmiadomainextractor: AhmiaDomainExtractor
```

Figure 3 – Create the transform script and insert it in the transforms folder

Access the transforms folder and insert the script, in this case it needs to have the same name as the Class created for the Transform, in my case it was AhmiaDomainExtractor. After that, run the Project.py script to see if the Transform script was detected.

```
$ python3 project.py list
```

The above command checks the list of projects

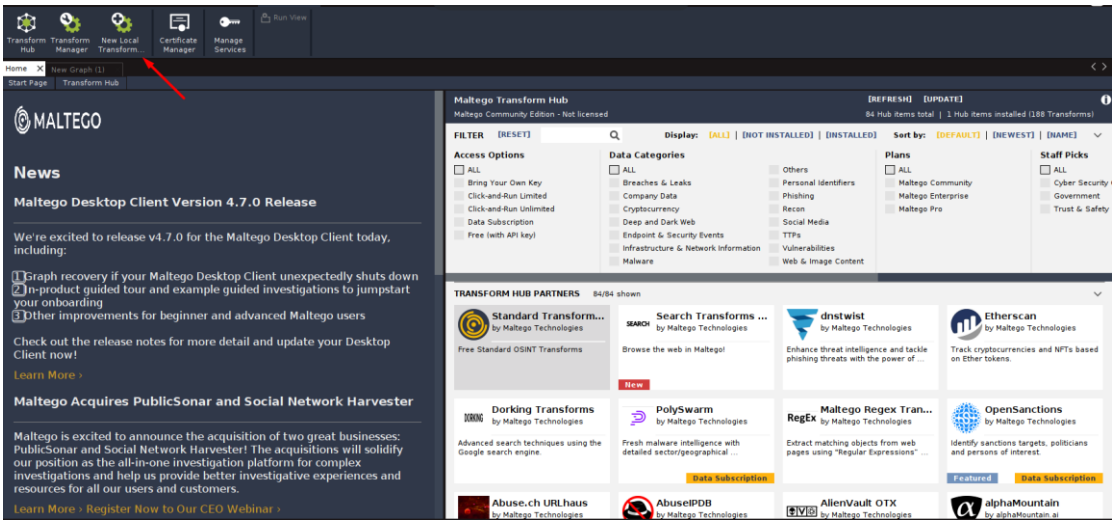

Figure 4 – Configuring a new transform

When accessing Maltego, click on the New Local Transform option

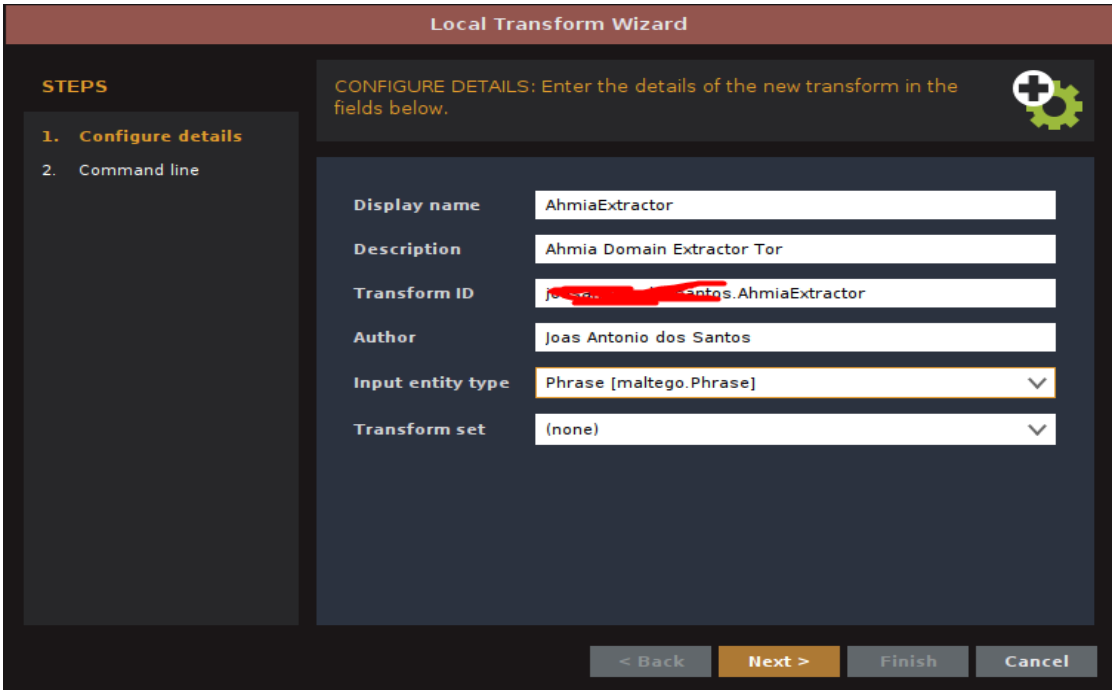

Figure 5 – Local Transform Configuration

When creating and managing Transforms in Maltego, several parameters and settings are important to define how the Transform operates and how it is presented in Maltego. Here's a summary of each of the terms you mentioned:

DisplayName

- **DisplayName**is the name of the Transform that is displayed in Maltego. It is used to identify Transform in the user interface, making it easier for users to quickly understand what Transform does. For example, a DisplayName could be "Search Ahmia for Domains".

Description

- **Description**provides a detailed description of what Transform does. This description helps the user to better understand the purpose of Transform, what it searches for, what type of data it returns, and any other relevant information that may help with its use.

Transform ID

- **Transform ID**is a unique identifier for Transform within Maltego. It is used to reference Transform programmatically and must be unique across the entire Maltego configuration. It is crucial for the integration and management of Transforms, especially when they are distributed across transformation servers or configured in shared environments.

Author

- **Author**refers to the creator or organization responsible for developing Transform. This field is useful for tracking, supporting and crediting features within Maltego.

Input Entity Type

- **Input Entity Type**specifies the type of entity that Transform expects as input. Each Transform in Maltego is designed to work with certain types of entities (such as domains, IP addresses, emails, etc.). This parameter defines which entities this specific Transform can interact with. For example, a Transform designed to extract information from a domain would expect an entity of type "Domain".

Transform Set

- **Transform Set**is a collection of Transforms grouped together in Maltego. Transform Sets are used to organize related Transforms into logical groups, making it easier for users to find and use Transforms that relate to specific tasks. For example, you might have a Transform Set called

"Domain Analysis" that includes several Transforms related to collection and analysisdomain data

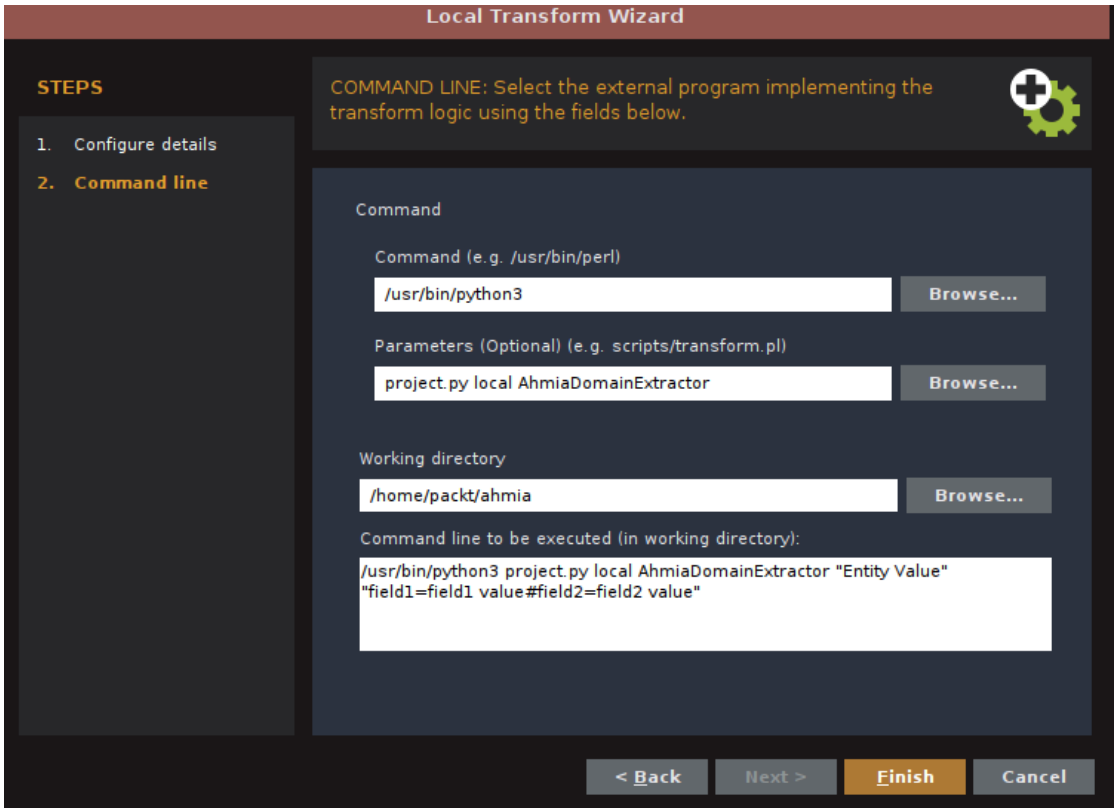

Figure 6 – Configuring the Command Line

Let's insert the Python interpreter in Command, in this case I'm on Linux and that's why I put the interpreter path /usr/bin/python

As parameters I define the local Project.py script "Transform Name" so that it starts the local transform when executed.

And finally the project directory where Transform is located.

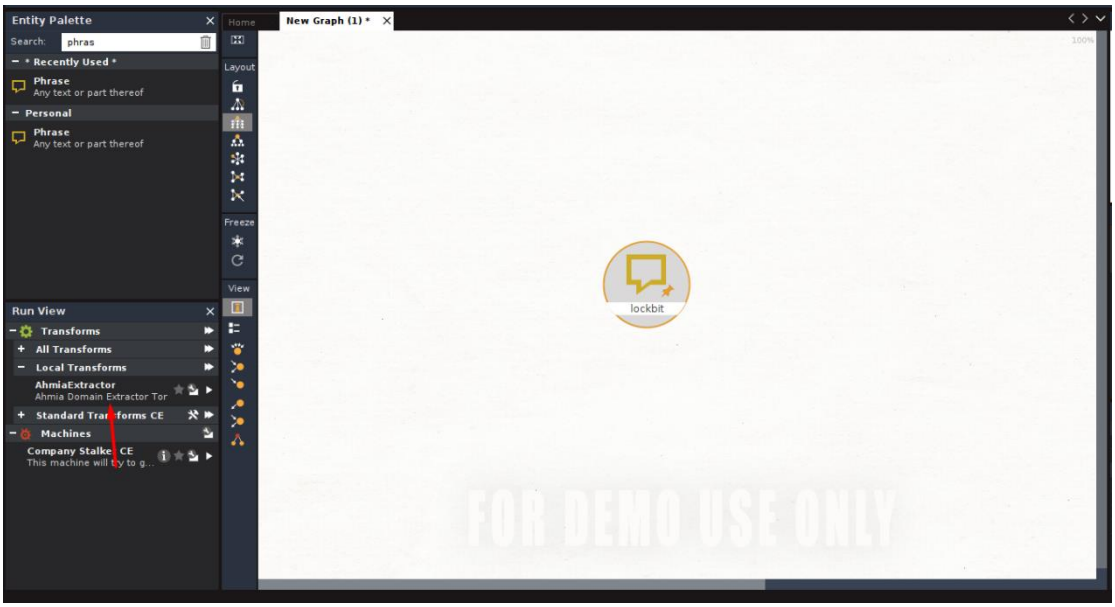

Figure 7 – Inserting a Phrase

After configuring our Transform, enter a phrase and enter a term you want to search, in this case we will use lockbit.

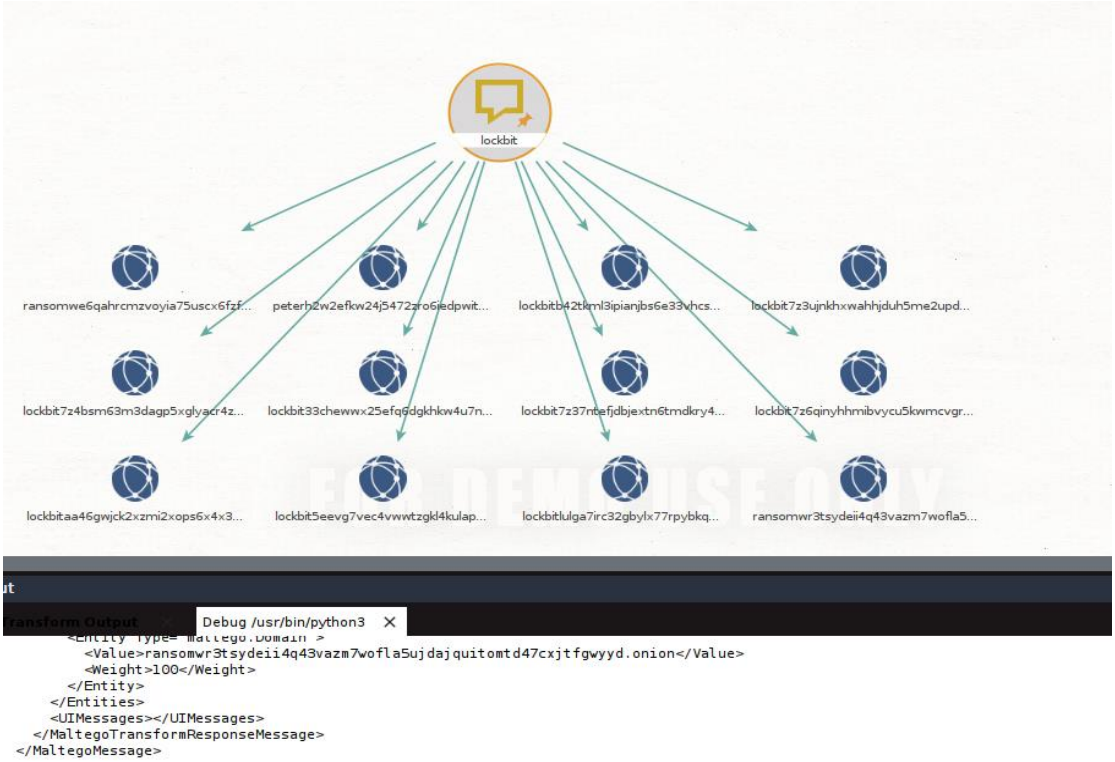

Figure 8 – Executing Transform

When running AhmiaDomainExtractor, this is the result, it brings all .onion domains related to the term lockbit.

Therefore, it is a Transform that becomes a complement to other Transforms that are used for investigations on the Dark Web

Conclusion

The Maltego tool is one of the most complete for carrying out OSINT, Intelligence and other types of investigation. This script is just an example of a project I am creating to further contribute to initiatives related to the investigation of Human Trafficking.

If you want to delve deeper into the development of Transforms and learn about other projects, I will leave some links below for your research.

https://www.youtube.com/watch?v=k5oikWy0OLc– Create your Local Transform by OSINT Dojo

https://github.com/cipher387/maltego-transforms-list- Maltego Transform List by Cipher387

https://github.com/megadose/holehe-maltego- Holehe Maltego by megadose

https://github.com/TURROKS/Maltego_WhatsMyName- WhatsMyName by TURROKS

https://www.youtube.com/watch?v=VRN741CgsOk&pp=ygUebG9jYWwgdHJhbnNmb3JtIG1hbHRlZ28gY3JlYXRl– Create your own local Transform in Python by Cylon Null

https://www.youtube.com/watch?v=42KhnNQS8AU- Official Maltego Tutorial – Writing your own Transforms

Bibliographic references

BUILDING INTEGRATIONS FOR MALTEGO. (nd). Retrieved May 12, 2024, fromhttps://static.maltego.com/cdn/Case%20studies/Building-Integrations-for-Maltego-Complete-Guide.pdf

Writing Transforms. (nd). Maltego Support. Retrieved May 12, 2024, fromhttps://docs.maltego.com/support/solutions/articles/15000015758-writing-transforms

ChatGPT 4 for translating and correcting texts